



Capital University of Science and Technology

Project Title

Automation Testing Framework for OpenCart Web Application

Submitted By

Student Name	Registration Number
Ali Akbar	BSE233016
Zainab Fatima	BSE233008
Laiba Zulfiqar	BSE233022

Submitted To

Syeda Hafsa Ali

Instructor, SE4343 - Automated Software Testing

Submission Date

28 May 2026

Table of Contents

Section 1: Introduction	5
1.1 Project Overview	5
1.2 Project Objectives	5
Section 2: System Description	6
2.1 System Under Test (OpenCart)	6
2.2 Key Features Tested	8
2.3 System Setup & Testing Constraints	8
Section 3: Test Strategy	11
3.1 Test Levels	11
3.2 Test Types	12
3.3 Tools Used	14
3.4 Test Environment	17
Section 4: Test Case Design	19
4.1 UI Test Cases (TC_001 to TC_010)	19
4.2 Unit Test Cases	21
4.4 Performance Test Cases	24
Section 5: Oracle Design	25
5.1 What is an Oracle?	25
5.2 Oracle Types Used	25
5.3 Oracle Examples by Test Category	26
5.3.1 Login Test Oracle	26
5.3.2 Search Test Oracle	26
5.3.3 Registration Negative Test Oracle	27

5.3.4 Cart Test Oracle.....	27
5.3.5 API Test Oracle	28
5.3.6 Property-Based Test Oracle	29
5.4 Oracle Design Summary	29
Section 6: Automation Implementation	30
6.1 Project Structure (POM - Page Object Model)	30
6.2 Framework Components	32
6.3 Key Code Snippets.....	33
6.3.1 BaseClass - Browser Setup	33
6.3.2 Page Object Example - RegisterPage	34
6.4 Test Execution Commands.....	35
Section 7: Test Execution Results	36
7.1 UI Test Results (Selenium + TestNG).....	36
7.2 Unit Test Results	39
7.4 Performance Test Results (JMeter)	40
7.5 Cross-Browser Test Configuration	42
Section 8: Flaky Test Handling	45
8.1 What Are Flaky Tests?.....	45
8.2 Identified Flaky Tests	45
8.3 Root Cause Analysis	46
8.4 Fixes Applied	47
8.5 Summary of Changes Made	50
8.6 Results After Fixes.....	50
8.7 Lessons Learned	51

Section 9: Test Adequacy Evaluation	52
9.1 What Was Tested (Functional Coverage)	52
9.2 Test Types Coverage	54
9.3 Test Count Summary	55
9.4 Bugs Found During Testing	56
9.5 What Was NOT Tested (Limitations)	57
9.6 Test Suite Adequacy Justification	58
9.7 Important Note	58
Section 10: Challenges & Lessons Learned	59
10.1 Technical Challenges Faced	59
10.2 Bugs Found During Testing	60
10.3 Key Takeaways	61
Section 11: Conclusion	63
11.1 Summary of Achievements	63
11.2 Final Test Results Summary	64
11.3 Bugs Discovered	64
11.4 Final Remarks	65

Section 1: Introduction

1.1 Project Overview

This project focuses on automated software testing of **OpenCart**, an open-source e-commerce platform. OpenCart allows users to browse products, add items to cart, register accounts, login, and complete checkout. The system was chosen because it provides realistic e-commerce workflows suitable for demonstrating automated testing techniques.

Why OpenCart was selected:

- Multiple user flows (registration, login, search, cart, checkout)
- Input variations (valid/invalid data, boundary values)
- Observable outputs (success messages, error alerts, page redirects)
- No CAPTCHA or restricted access
- Can be run locally using XAMPP

1.2 Project Objectives

The main objectives of this project are:

1. Apply automated software testing techniques to ensure system correctness and reliability
2. Implement different test levels: Unit, API, UI, and End-to-End
3. Cover various test types: Functional, Boundary, Negative, and Property-Based
4. Design clear oracles (what is being checked and why it is correct)
5. Automate tests using industry-standard tools
6. Handle flaky tests and ensure stable execution
7. Evaluate test adequacy and document limitations

1.3 Document Structure

This report is organized into 12 sections:

Section	Content
1-2	Introduction and System Description
3-5	Test Strategy, Test Cases, Oracle Design
6-7	Automation Implementation and Test Results
8-9	Flaky Test Handling and Test Adequacy
10-12	Challenges, Conclusion, and Appendices

Table 1: All Sections

Section 2: System Description

2.1 System Under Test (OpenCart)

OpenCart is a free, open-source e-commerce platform used by thousands of online stores worldwide. It is written in PHP and uses MySQL as its database. For this project, OpenCart version 4.x was installed locally using XAMPP and accessed at:

<http://localhost/opencart/>

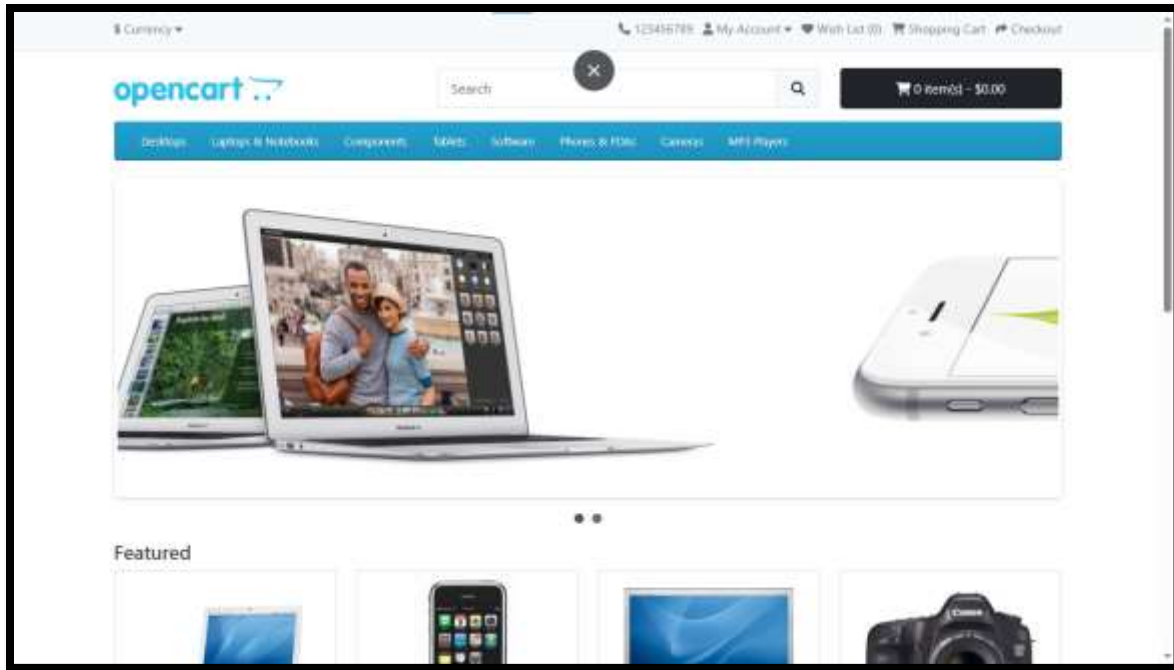


Figure 1: OpenCart Home Page

Key characteristics of the system:

- User-friendly frontend for customers
- Admin backend for store management
- Supports product browsing, searching, cart management, and checkout
- Built-in customer registration and login system

2.2 Key Features Tested

The following core features were selected for automation testing:

Feature	Description	Test Coverage
User Registration	New user signup with email and password	TC_001, TC_006
User Login	Authentication with valid/invalid credentials	TC_002
Product Search	Search functionality with various inputs	TC_003, TC_009
My Account	Account management links (edit, password, wishlist, orders)	TC_004
Shopping Cart	Add, remove, update quantity, view cart	TC_005
Checkout Process	Proceed to checkout, cart total display	TC_007, TC_010
Product Page	Display product details, image, price, quantity	TC_008

Table 2: Key Features Tested

2.3 System Setup & Testing Constraints

System Setup (Actually Implemented):

Item	Details
OpenCart Installation	Downloaded and installed manually
Web Server	XAMPP (Apache) configured and running
Database	MySQL configured with OpenCart
Access URL	http://localhost/opencart/
Sample Products	Mac, iPhone, Apple, HP, Canon (available in catalog)

Table 3: System Setup & Testing Constraints

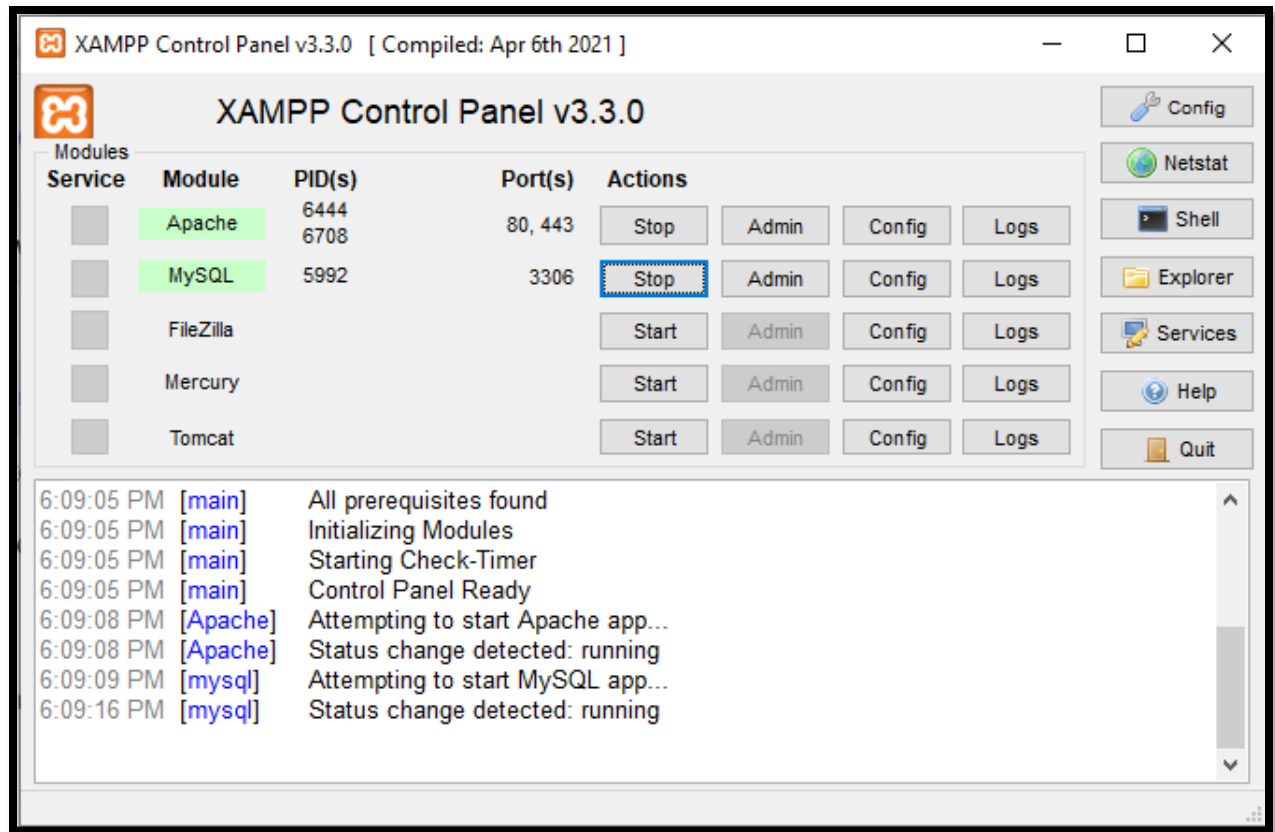


Figure 2: Xampp

Testing Constraints (Limitations):

Constraint	Impact	Reason
Payment gateway not configured	Cannot complete actual order placement	Local environment, no payment module setup
Email server not configured	Cannot test email confirmation flows	Local XAMPP has no mail server
Admin panel not tested	Focus only on customer-facing features	Project scope limitation

Order placement not executed	E2E test stops at checkout page	No payment method available
-------------------------------------	---------------------------------	-----------------------------

Table 4: Testing Constraints (Limitations):

What Was NOT Assumed (Actually Done):

- OpenCart was downloaded and installed (not assumed to exist)
- XAMPP was configured and running (not assumed)
- Database was set up with sample data (not assumed)
- Products exist and are searchable (verified)

Section 3: Test Strategy

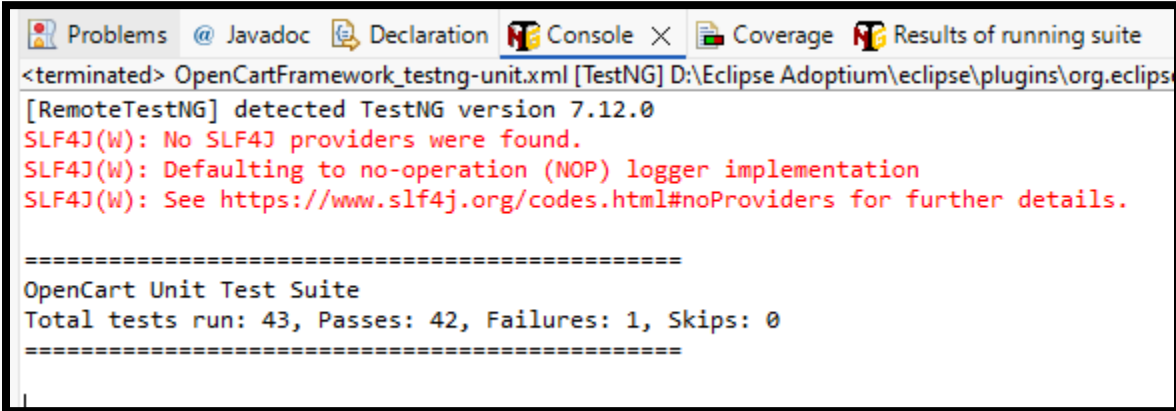
3.1 Test Levels

The following test levels were implemented in this project:

Test Level	Implemented	Description
Unit Testing	Yes	Testing individual validation methods (email, password, quantity, checkout math)
API Testing	Yes	Testing REST API endpoints using Postman + Newman
UI Testing	Yes	Testing web interface using Selenium WebDriver
End-to-End Testing	Yes	Complete user journey from registration to checkout

Table 5: Test Levels

For unit Testing



```
<terminated> OpenCartFramework_testng-unit.xml [TestNG] D:\Eclipse Adoptium\eclipse\plugins\org.eclipse
[RemoteTestNG] detected TestNG version 7.12.0
SLF4J(W): No SLF4J providers were found.
SLF4J(W): Defaulting to no-operation (NOP) logger implementation
SLF4J(W): See https://www.slf4j.org/codes.html#noProviders for further details.

=====
OpenCart Unit Test Suite
Total tests run: 43, Passes: 42, Failures: 1, Skips: 0
=====
```

Figure 3: Unit Tests Console Result

Add the whole suit



Figure 4: Ui Test Result ExtentReport

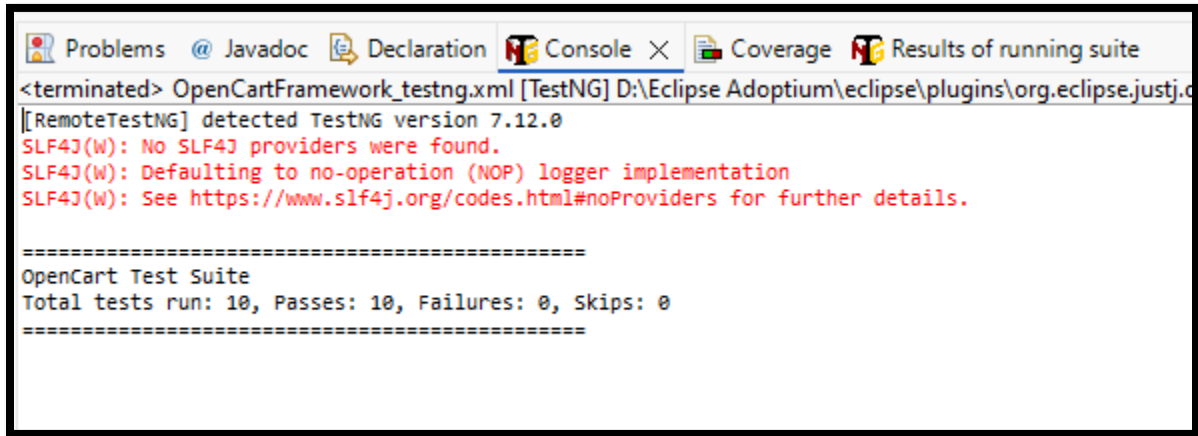
3.2 Test Types

The following test types were covered:

Test Type	Implemented	Evidence
Functional Testing	Yes	All TC_XXX files test core functionality
Boundary Testing	Yes	Password min/max length, search length, quantity limits
Negative Testing	Yes	Invalid login, empty form, duplicate email, invalid search
Property-Based Testing	Yes	Case-insensitive search (invariant property)

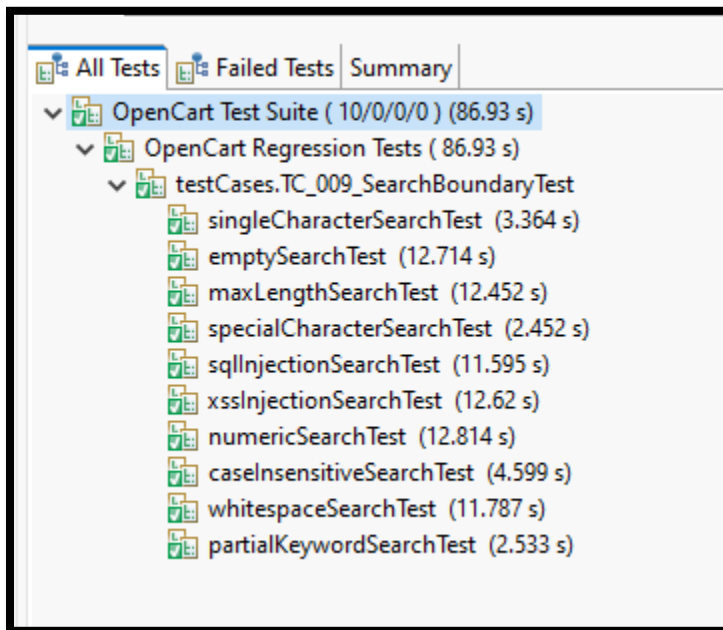
Table 6: Test Types

Boundary Tests Screenshots



The screenshot shows the Eclipse IDE's Console window. The title bar includes tabs for Problems, Javadoc, Declaration, Console (active), Coverage, and Results of running suite. The console output shows the termination of the OpenCartFramework_testng.xml [TestNG] suite. It reports that RemoteTestNG detected TestNG version 7.12.0 and that no SLF4J providers were found, defaulting to a no-operation (NOP) logger implementation. The test suite summary shows 10 tests run, all passed, with 0 failures and 0 skips.

```
<terminated> OpenCartFramework_testng.xml [TestNG] D:\Eclipse Adoptium\ eclipse\plugins\org.eclipse.justj.c  
[RemoteTestNG] detected TestNG version 7.12.0  
SLF4J(W): No SLF4J providers were found.  
SLF4J(W): Defaulting to no-operation (NOP) logger implementation  
SLF4J(W): See https://www.slf4j.org/codes.html#noProviders for further details.  
  
=====  
OpenCart Test Suite  
Total tests run: 10, Passes: 10, Failures: 0, Skips: 0  
=====
```



The screenshot shows the Eclipse IDE's Test Explorer window. The title bar includes tabs for All Tests (active), Failed Tests, and Summary. The test suite structure is displayed as follows:

- OpenCart Test Suite (10/0/0/0) (86.93 s)
 - OpenCart Regression Tests (86.93 s)
 - testCases.TC_009_SearchBoundaryTest
 - singleCharacterSearchTest (3.364 s)
 - emptySearchTest (12.714 s)
 - maxLengthSearchTest (12.452 s)
 - specialCharacterSearchTest (2.452 s)
 - sqlInjectionSearchTest (11.595 s)
 - xssInjectionSearchTest (12.62 s)
 - numericSearchTest (12.814 s)
 - caseInsensitiveSearchTest (4.599 s)
 - whitespaceSearchTest (11.787 s)
 - partialKeywordSearchTest (2.533 s)

3.3 Tools Used

The following tools were selected for automation:

Tool	Purpose	Version
Selenium WebDriver	UI automation	4.44.0
TestNG	Test framework and assertions	7.12.0
Java	Programming language	11
Maven	Build and dependency management	3.x
ExtentReports	HTML reporting	5.1.2
Apache POI	Excel data handling	5.4.1
Postman + Newman	API testing	Newman 6.2.2
JMeter	Performance testing	5.x
XAMPP	Local web server	Latest

Table 7: Tools Used

All Dependencies

```
17
18- <dependencies>
19
20     <!-- Selenium Java -->
21- <dependency>
22         <groupId>org.seleniumhq.selenium</groupId>
23         <artifactId>selenium-java</artifactId>
24         <version>4.44.0</version>
25     </dependency>
26
27     <!-- TestNG -->
28- <dependency>
29         <groupId>org.testng</groupId>
30         <artifactId>testng</artifactId>
31         <version>7.12.0</version>
32         <scope>test</scope>
33     </dependency>
34
35     <!-- Apache POI -->
36- <dependency>
37         <groupId>org.apache.poi</groupId>
38         <artifactId>poi-ooxml</artifactId>
39         <version>5.4.1</version>
40     </dependency>
41
42     <!-- Extent Reports -->
43- <dependency>
44         <groupId>com.aventstack</groupId>
45         <artifactId>extentreports</artifactId>
46         <version>5.1.2</version>
47     </dependency>
48
49     <!-- Log4j API -->
50- <dependency>
51         <groupId>org.apache.logging.log4j</groupId>
52         <artifactId>log4j-api</artifactId>
53         <version>2.26.0</version>
54     </dependency>
55
```

```
56      <!-- Log4j Core -->
57      <dependency>
58          <groupId>org.apache.logging.log4j</groupId>
59          <artifactId>log4j-core</artifactId>
60          <version>2.26.0</version>
61      </dependency>
62
63  </dependencies>
64
65  <build>
66
67      <plugins>
68
69          <!-- Maven Compiler Plugin -->
70          <plugin>
71              <groupId>org.apache.maven.plugins</groupId>
72              <artifactId>maven-compiler-plugin</artifactId>
73              <version>3.11.0</version>
74
75              <configuration>
76                  <source>11</source>
77                  <target>11</target>
78              </configuration>
79          </plugin>
80
81          <!-- Maven Surefire Plugin -->
82          <plugin>
83              <groupId>org.apache.maven.plugins</groupId>
84              <artifactId>maven-surefire-plugin</artifactId>
85              <version>3.1.2</version>
86
87              <configuration>
88                  <suiteXmlFiles>
89                      <suiteXmlFile>testng.xml</suiteXmlFile>
90                  </suiteXmlFiles>
91              </configuration>
92          </plugin>
93
94      </plugins>
95
96  </build>
```


3.4 Test Environment

Hardware:

Component	Specification
Operating System	Windows 10
RAM	8 GB
Processor	Intel Core i7

Table 8: Hardware

Software:

Component	Version
Web Browser	Chrome (primary), Edge, Firefox (configured)
Web Driver	ChromeDriver, EdgeDriver, GeckoDriver
IDE	Eclipse
Local Server	XAMPP (Apache + MySQL)
Application	OpenCart 4.x

Table 9: Software

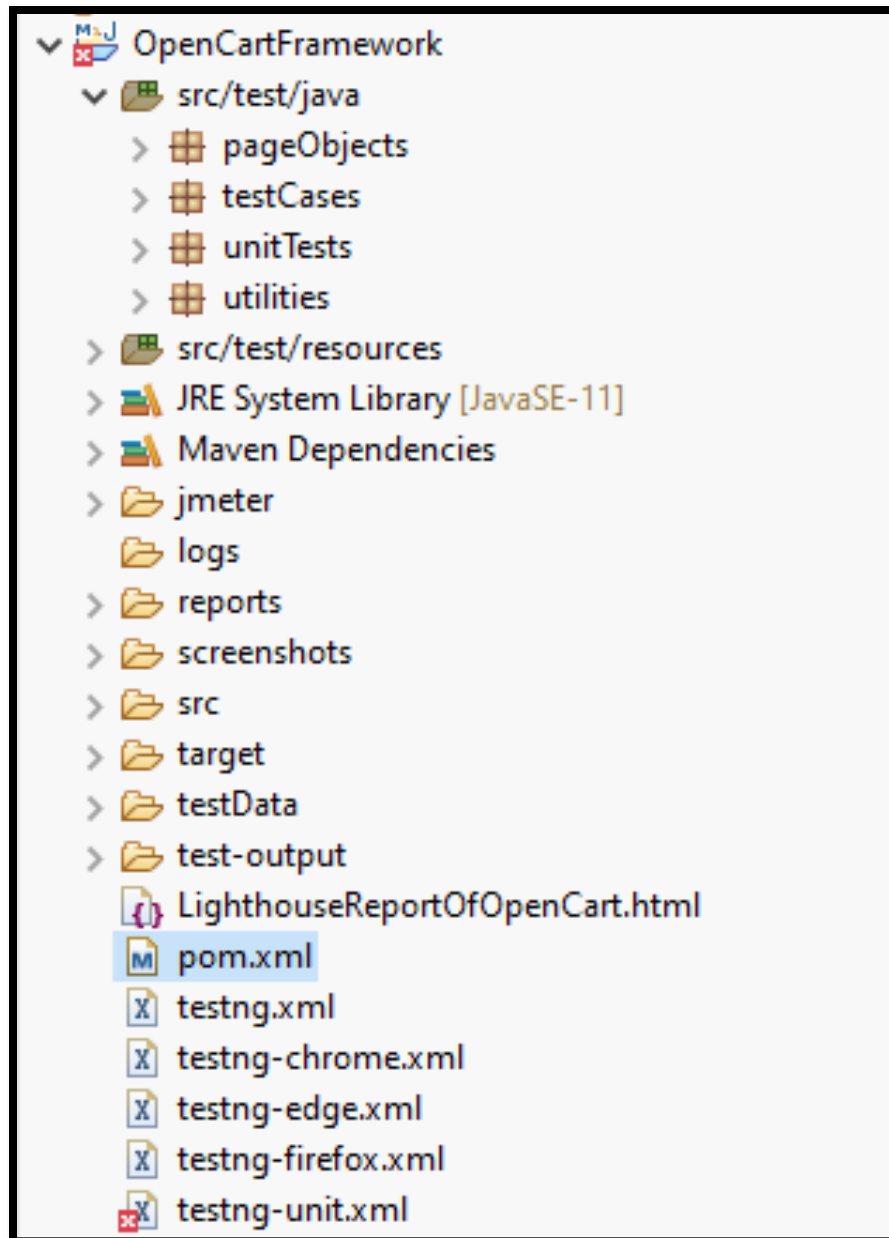


Figure 5: Folder Structure Of OpenCart

Section 4: Test Case Design

4.1 UI Test Cases (TC_001 to TC_010)

A total of 10 UI test classes were created covering all major functionalities of OpenCart. Each test case targets a specific condition with clear expected outputs.

Test ID	Test Class Name	Description	Test Type
TC_001	AccountRegistrationTest	Valid user registration with unique email	Functional
TC_002	LoginTest	Valid and invalid login with data-driven approach	Functional + Negative
TC_003	SearchTest	Product search with valid and invalid keywords	Functional
TC_004	MyAccountTest	Account page links (edit, password, wishlist, orders, logout)	Functional
TC_005	CartTest	Add, remove, update quantity, view cart	Functional
TC_006	RegisterNegativeTest	Registration with invalid data (empty, duplicate, boundary)	Negative + Boundary
TC_007	CheckoutTest	Guest checkout, logged-in checkout, cart total, quantity update	Functional + E2E
TC_008	ProductPageTest	Product details, image, price, quantity (zero and negative)	Functional + Negative
TC_009	SearchBoundaryTest	Search with single char, empty, max length, special chars, SQL, XSS	Boundary + Security

TC_010	EndToEndTest	Complete user journey: Register → Login → Search → Add to Cart → Checkout	E2E
--------	--------------	---	-----

Table 10: UI Test Cases (TC_001 to TC_010)

TestNg-Chrome.xml

```

testng-chrome.xml X
http://testng.org/testng-1.9.8.dtd (doctype)
1  <?xml version="1.0" encoding="UTF-8"?>
2  <!DOCTYPE suite SYSTEM "http://testng.org/testng-1.9.8.dtd">
3
4  <suite name="OpenCart - Chrome Browser Tests">
5
6      <listeners>
7          <listener class-name="utilities.ExtentReportManager"/>
8      </listeners>
9
10     <test name="Chrome - Full Regression Suite">
11         <parameter name="browser" value="chrome"/>
12
13         <groups>
14             <run>
15                 <include name="regression"/>
16             </run>
17         </groups>
18
19         <classes>
20             <class name="testCases.TC_001_AccountRegistrationTest"/>
21             <class name="testCases.TC_002_LoginTest"/>
22             <class name="testCases.TC_003_SearchTest"/>
23             <class name="testCases.TC_004_MyAccountTest"/>
24             <class name="testCases.TC_005_CartTest"/>
25             <class name="testCases.TC_006_RegisterNegativeTest"/>
26             <class name="testCases.TC_007_CheckoutTest"/>
27             <class name="testCases.TC_008_ProductPageTest"/>
28             <class name="testCases.TC_009_SearchBoundaryTest"/>
29             <class name="testCases.TC_010_EndToEndTest"/>
30         </classes>
31     </test>
32
33 </suite>

```

Figure 6: TestNg-Chrome.xml

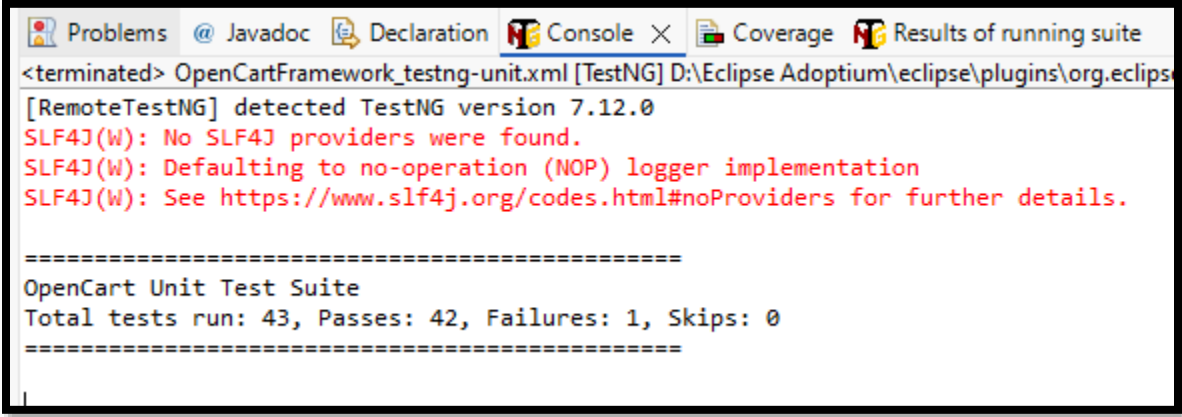
4.2 Unit Test Cases

Unit tests were created to validate business logic in isolation without browser or database dependencies.

Unit Test Class	What It Tests	Number of Tests
EmailValidatorTest	Email format validation (null, empty, missing @, missing dot, spaces, max length)	12
PasswordValidatorTest	Password length (4-20 chars) and strength classification (WEAK/MEDIUM/STRONG)	13
ProductQuantityValidatorTest	Quantity rules (positive, max 9999, zero, negative)	10
CheckoutTotalCalculatorTest	Cart math (subtotal, tax, shipping, additive property)	13

Table 11: Unit Test Cases

Total Unit Tests: 43



```
<terminated> OpenCartFramework_testng-unit.xml [TestNG] D:\Eclipse Adoptium\eclipse\plugins\org.eclipse
[RemoteTestNG] detected TestNG version 7.12.0
SLF4J(W): No SLF4J providers were found.
SLF4J(W): Defaulting to no-operation (NOP) logger implementation
SLF4J(W): See https://www.slf4j.org/codes.html#noProviders for further details.

=====
OpenCart Unit Test Suite
Total tests run: 43, Passes: 42, Failures: 1, Skips: 0
=====
```

Figure 7: Unit Tests Result

4.3 API Test Cases

API testing was performed using Postman and Newman. The following endpoints were tested:

Module	Endpoint	Test Focus
CRUD Operations	POST /api/products	Create product
CRUD Operations	GET /api/products	Read all products
CRUD Operations	GET /api/products/:id	Read single product
CRUD Operations	PUT /api/products/:id	Update product
CRUD Operations	DELETE /api/products/:id	Delete product
Negative Testing	Various endpoints	Invalid ID, missing fields, invalid price
Authentication	POST /api/auth/login	Valid and invalid login, token generation
Order Management	POST /api/orders	Create order with valid/invalid data
Order Management	GET /api/orders	Retrieve all orders

Table 12:API Test Cases

Postman OpenCart Collection

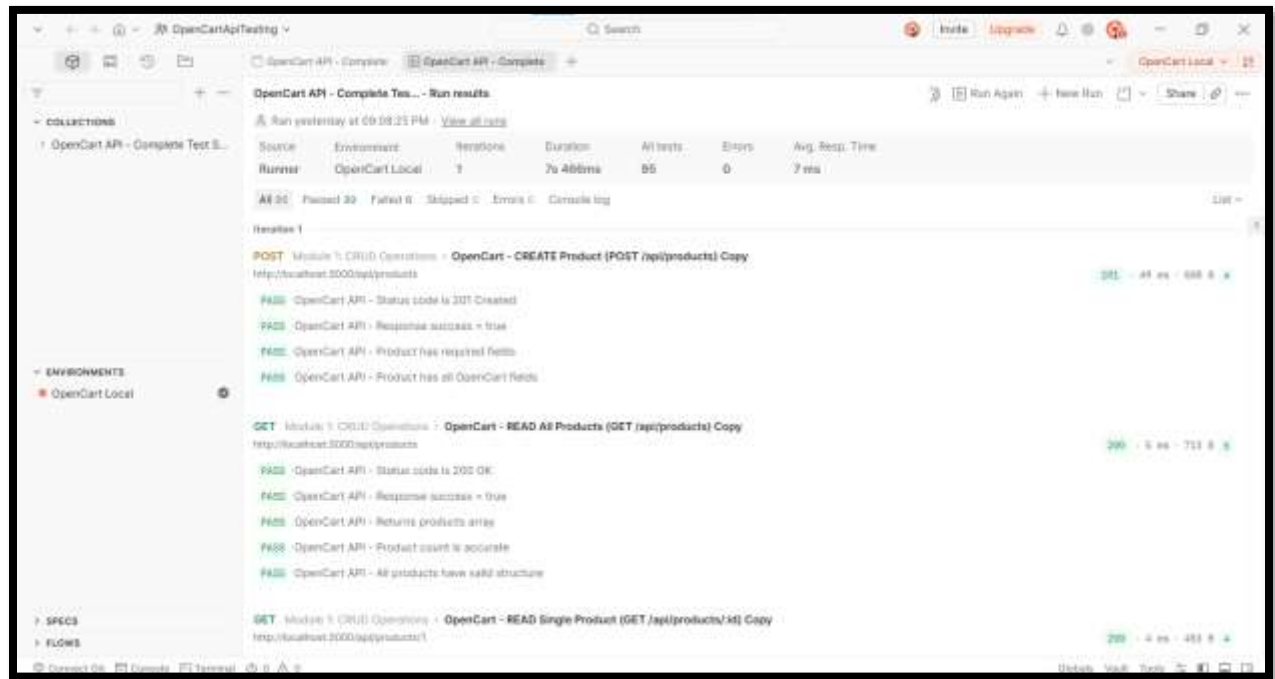


Figure 8: Postman Api Testing Result

4.4 Performance Test Cases

JMeter was used to create three performance test scenarios:

Test File	Purpose	Users	Duration
OpenCart Homepage Load Test	Measure homepage response time	10 concurrent	60 seconds
OpenCart Login Stress Test	Test login under load	20 concurrent	60 seconds
OpenCart Search Performance Test	Measure search response time	15 concurrent	60 seconds

Table 13: Performance Test Cases

Jmeter Performance Testing Result

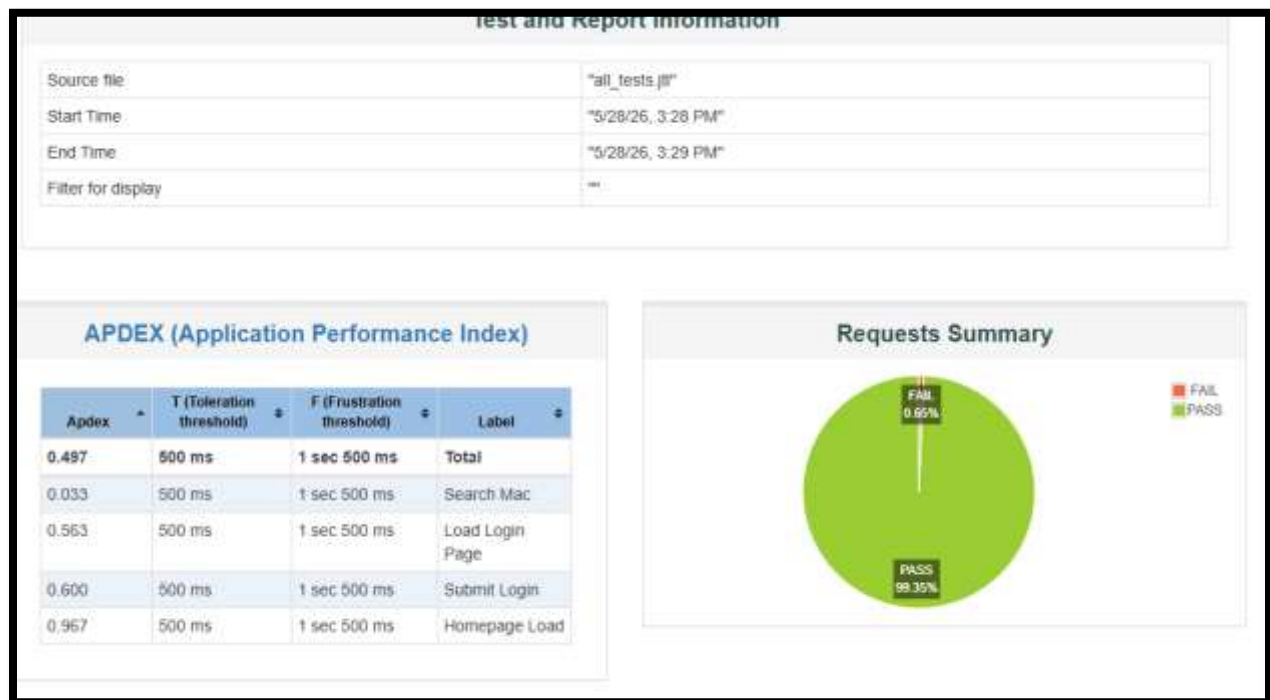


Figure 9: Jmeter Performance Testing Result

Section 5: Oracle Design

5.1 What is an Oracle?

An oracle is the mechanism used to determine whether a test has passed or failed. It answers two questions:

1. **What is being checked?** - The specific behavior or output being verified
2. **Why is it correct?** - The expected behavior based on system requirements or business rules

Every automated test in this project clearly states these two elements.

5.2 Oracle Types Used

Oracle Type	Definition	Example from Project
Expected Value Comparison	Compare actual output with expected value	Assert.assertEquals(quantity, "2")
Invariant Check	Verify a property that always holds true	Cart total >= 0, Password length 4-20 chars
State Validation	Verify system state change after action	Login -> MyAccount page displayed
API Response Validation	Verify status codes and response schema	Postman: 200 OK, JSON structure
Error Message Validation	Verify correct error appears	"Warning: No match for E-Mail"

Table 14: Oracle Types Used

```

String totalText = checkoutPage.getCartTotal();
Assert.assertTrue(totalText.contains("$"), "Cart total should display dollar amount");
logInfo("✓ Checkout cart total: " + totalText);

// ===== TEST COMPLETE =====
logInfo("=====");
logInfo("===== END-TO-END TEST COMPLETED SUCCESSFULLY =====");
logInfo("=====");
logInfo("User journey verified:");
logInfo("  ✓ Register → Login → Search → Add to Cart → View Cart → Checkout");
logInfo("=====");
logInfo("TEST PASSED: completeUserJourneyTest (E2E)");
logInfo("=====");

```

Figure 10: Oracle Example

5.3 Oracle Examples by Test Category

5.3.1 Login Test Oracle

What is being checked	Why it is correct
MyAccount page appears after valid credentials	OpenCart redirects authenticated users to account dashboard
Warning message appears for invalid credentials	OpenCart displays "Warning: No match for E-Mail Address and/or Password" when authentication fails
Browser shows validation error for malformed email	HTML5 specification requires email fields to contain '@' symbol

Table 15: Login Test Oracle

5.3.2 Search Test Oracle

What is being checked	Why it is correct
Search returns at least one product	OpenCart returns products matching the query from database

"No results" message appears for non-existent product	OpenCart displays "There is no product that matches the search criteria"
--	--

Table 16: Search Test Oracle

5.3.3 Registration Negative Test Oracle

What is being checked	Why it is correct
Password below minimum length rejected	OpenCart minimum password length is 4 characters
Email without @ is rejected	Email must follow RFC 5322 format (contains @ and domain)
Already registered email rejected	OpenCart requires unique email addresses per account

Table 17:Registration Negative Test Oracle

5.3.4 Cart Test Oracle

What is being checked	Why it is correct
Cart count updates after adding product	OpenCart cart component shows "1 item(s)" immediately after adding
Cart becomes empty after removal	Cart should display "Your shopping cart is empty!" message

Table 18:Cart Test Oracle

5.3.5 API Test Oracle

What is being checked	Why it is correct
Status code is 201 Created	POST request successfully created a resource
Status code is 200 OK	GET/PUT request completed successfully
Status code is 400 Bad Request	Invalid input was properly rejected
Status code is 401 Unauthorized	Invalid credentials were rejected
Status code is 404 Not Found	Non-existent resource returns proper error

Table 19: API Test Oracle

```
1 // OpenCart API Test: Create new product
2
3 // Test 1: Status code is 201 (Created)
4 pm.test("OpenCart API - Status code is 201 Created", function () {
5     pm.response.to.have.status(201);
6 });
7
8 // Test 2: Response has success flag
9 pm.test("OpenCart API - Response success = true", function () {
10     var jsonData = pm.response.json();
11     pm.expect(jsonData.success).to.eql(true);
12 });
13
14 // Test 3: Product data is returned with OpenCart structure
15 pm.test("OpenCart API - Product has required fields", function () {
16     var jsonData = pm.response.json();
17     pm.expect(jsonData.data).to.be.an('object');
18     pm.expect(jsonData.data.id).to.be.a('number');
19     pm.expect(jsonData.data.name).to.be.a('string');
20     pm.expect(jsonData.data.price).to.be.a('number');
21 });
```

Figure 11: API Oracle Example

5.3.6 Property-Based Test Oracle

Property	Test Method	Invariant Checked
Case Insensitivity	caseInsensitiveSearchTest	searchResults("IMAC") == searchResults("imac") for all queries
Non-Negative Total	testTotalProperty_AlwaysNonNegative	Cart total >= 0 always
Quantity Range	testQuantityProperty_ValidRangeOnly	Valid quantity is between 1 and 9999
Strength Consistency	testStrengthConsistentProperty	Multiple calls to getStrength() return same result

Table 20: Property-Based Test Oracle

```
Assert.assertTrue(currentUrl.contains("checkout") || currentUrl.contains("login"),
    "Guest should be redirected to checkout or login page");

logInfo("Guest checkout redirect working. Current URL: " + currentUrl);
logInfo("TEST PASSED: guestCheckoutRedirectTest");
logInfo("=====");
```

5.4 Oracle Design Summary

All automated tests in this project implement clear oracles that answer:

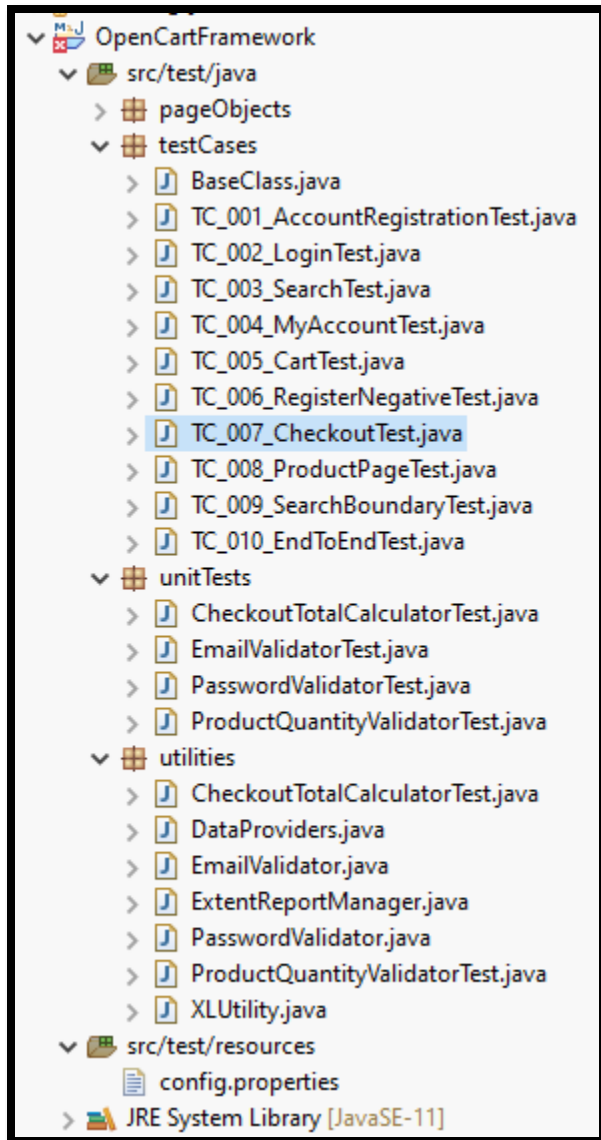
1. **What is being checked** - specific behavior or output
2. **Why it is correct** - based on OpenCart's documented behavior, business rules, or mathematical invariants

The combination of expected value comparison, state validation, error message validation, and property-based invariants provides comprehensive verification of system correctness.

Section 6: Automation Implementation

6.1 Project Structure (POM - Page Object Model)

The project follows the Page Object Model (POM) design pattern. This approach separates test logic from page-specific code, making the framework maintainable and reusable.



- >  Maven Dependencies
- ▼  jmeter
 - >  combined-report
 -  all_tests.jtl
 -  jmeter.log
 -  OpenCart Homepage Load Test.jmx
 -  OpenCart Login Stress Test.jmx
 -  OpenCart Search Performance Test.jmx
 -  run.bat
 -  logs
 - >  reports
 - >  screenshots
 - >  src
 - >  target
 - >  testData
 - >  test-output
 -  LighthouseReportOfOpenCart.html
 -  pom.xml
 -  testng.xml
 -  testng-chrome.xml
 -  testng-edge.xml
 -  testng-firefox.xml
 -  testng-unit.xml

6.2 Framework Components

Component	Purpose	Key Files
Page Objects	Locate and interact with web elements	CartPage, LoginPage, RegisterPage, etc.
Test Cases	Test logic and assertions	TC_001 to TC_010
BaseClass	Common setup, teardown, browser management	BaseClass.java
Utilities	Data providers, reporting, Excel handling	DataProviders, ExtentReportManager, XLUtility
Config Properties	Environment and test data configuration	config.properties
Unit Tests	Business logic validation	EmailValidatorTest, etc.

6.3 Key Code Snippets

6.3.1 BaseClass - Browser Setup

```
@BeforeMethod(groups = { "sanity", "smoke", "regression", "master" })
@Parameters({ "browser" })
public void setup(String browser) throws IOException {

    // load config.properties
    FileReader file = new FileReader(System.getProperty("user.dir") + "\\src\\test\\resources\\config.properties");
    p = new Properties();
    p.load(file);

    // launch browser based on parameter
    switch (browser.toLowerCase()) {
        case "chrome":
            driver = new ChromeDriver();
            break;
        case "edge":
            driver = new EdgeDriver();
            break;
        case "firefox":
            driver = new FirefoxDriver();
            break;
        default:
            System.out.println("Invalid browser: " + browser + " - launching Chrome");
            driver = new ChromeDriver();
    }

    driver.manage().deleteAllCookies();
    driver.manage().timeouts().implicitlyWait(Duration.ofSeconds(10));
    driver.manage().window().maximize();
    driver.get(p.getProperty("appURL"));
}
```

6.3.2 Page Object Example - RegisterPage

```
BaseClass.java RegisterPage.java X
1 package pageObjects;
2
3 import java.time.Duration;
12
13 public class RegisterPage {
14
15     WebDriver driver;
16     WebDriverWait wait;
17
18     @FindBy(id = "input-firstname")
19     WebElement firstNameField;
20
21     @FindBy(id = "input-lastname")
22     WebElement lastNameField;
23
24     @FindBy(id = "input-email")
25     WebElement emailField;
26
27     @FindBy(id = "input-telephone")
28     WebElement telephoneField;
29
30     @FindBy(id = "input-password")
31     WebElement passwordField;
32
33     @FindBy(id = "input-confirm")
34     WebElement confirmPasswordField;
35
36     @FindBy(id = "input-newsletter")
37     WebElement newsletter;
38
39     @FindBy(css = "input[name='agree']")
40     WebElement privacyPolicyCheckbox;
41
42     @FindBy(xpath = "//button[normalize-space()='Continue']")
43     WebElement continueButton;
44
45     @FindBy(xpath = "//h1[normalize-space()='Your Account Has Been Created!']")
46     WebElement successHeading;
47
```

```

232
233 public boolean isPasswordErrorDisplayed() {
234     try {
235         wait.until(ExpectedConditions.visibilityOf(passwordError));
236         return passwordError.isDisplayed();
237     } catch (Exception e) {
238         return false;
239     }
240 }
241
242 // ----- FULL FLOW -----
243
244 public void register(String firstName, String lastName, String email, String telephone, String password,
245                     boolean newsletterFlag) {
246     enterFirstName(firstName);
247     enterLastName(lastName);
248     enterEmail(email);
249     enterTelephone(telephone);
250     enterPassword(password);
251     enterConfirmPassword(password);
252     selectNewsletter(newsletterFlag);
253     acceptPrivacyPolicy();
254     clickContinue();
255 }
256
257 // ----- SAFE CLICK UTILITY -----
258
259 private void scrollAndClick(WebElement element) {
260     try {
261         ((JavascriptExecutor) driver).executeScript("arguments[0].scrollIntoView(true);", element);
262         element.click();
263     } catch (Exception e) {
264         ((JavascriptExecutor) driver).executeScript("arguments[0].click();", element);
265     }
266 }
267 }

```

6.4 Test Execution Commands

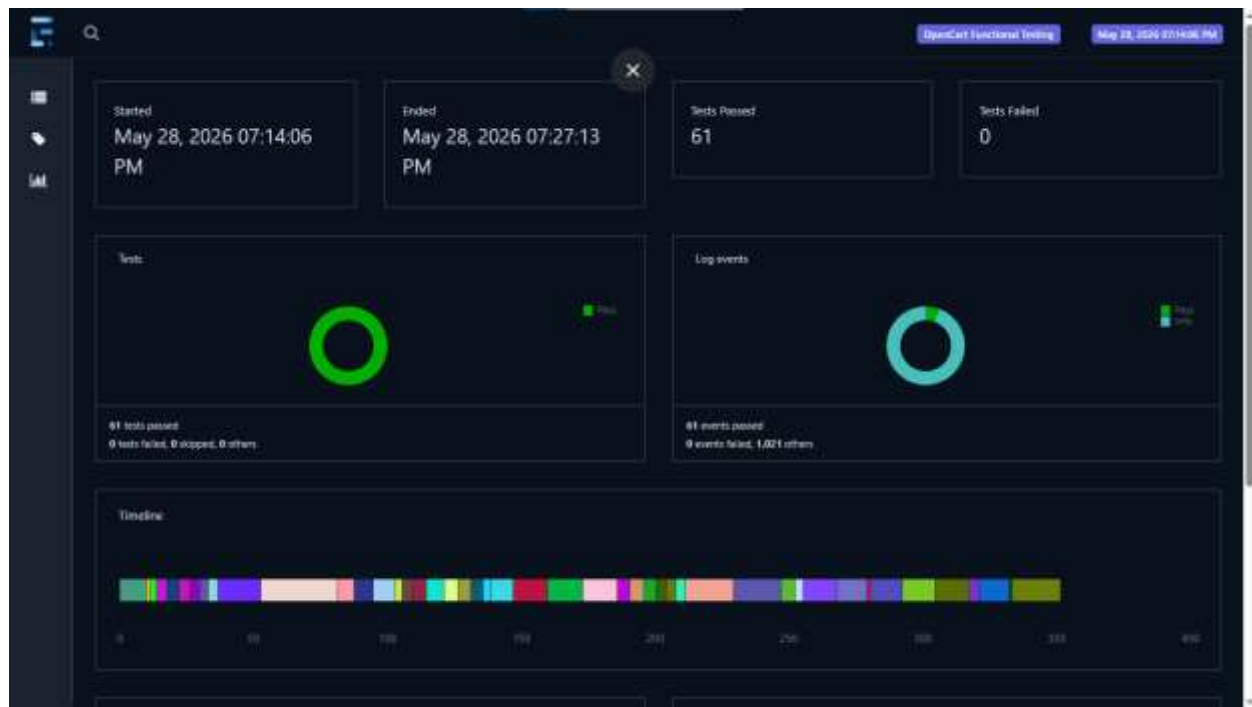
Test Type	Command
Run all UI tests (Chrome)	mvn test -Dsurefire.suiteXmlFiles=testng-chrome.xml
Run all UI tests (Edge)	mvn test -Dsurefire.suiteXmlFiles=testng-edge.xml
Run all UI tests (Firefox)	mvn test -Dsurefire.suiteXmlFiles=testng-firefox.xml
Run only unit tests	mvn test -Dtest=unitTests.*
Run specific test class	mvn test -Dtest=TC_002_LoginTest
Run API tests (Newman)	newman run OpenCart-API-Collection.json
Run JMeter tests	jmeter -n -t test.jmx -l results.jtl -e -o report

Section 7: Test Execution Results

7.1 UI Test Results (Selenium + TestNG)

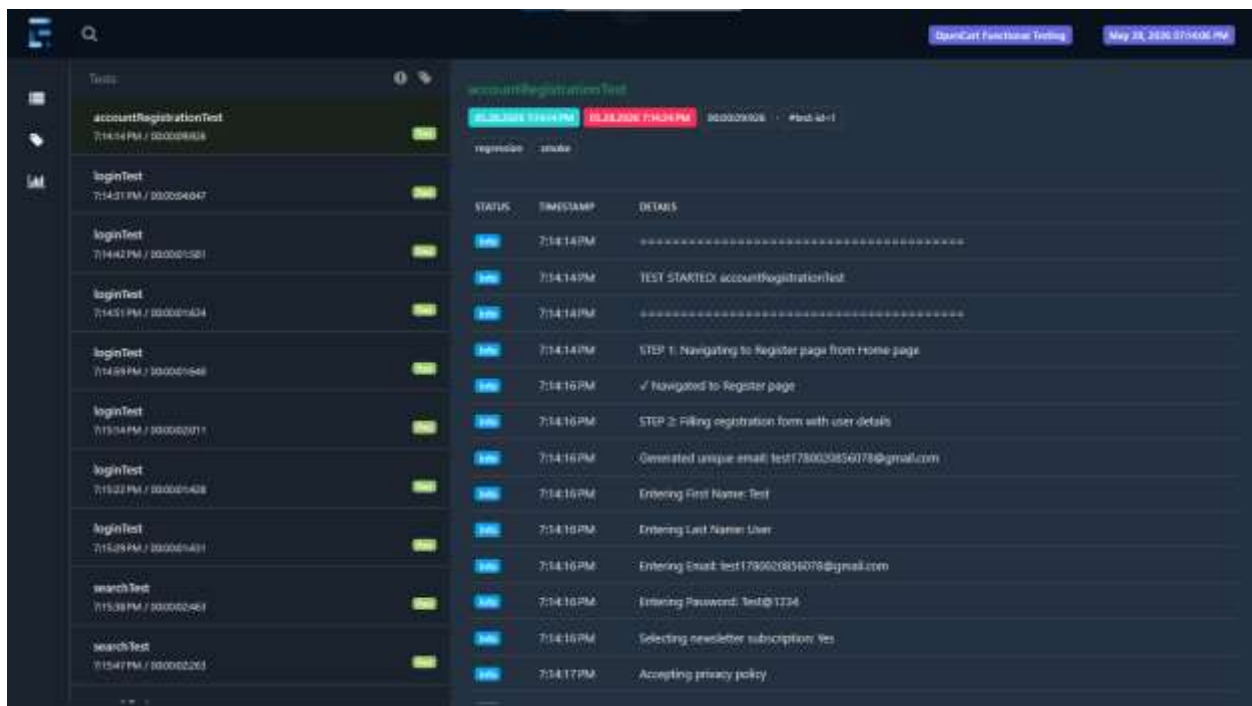
A total of 61 UI tests were executed covering registration, login, search, cart, checkout, and product page functionalities. All tests passed successfully.

Test Class	Number of Tests	Passed	Failed	Pass Rate
TC_001_AccountRegistrationTest	1	1	0	100%
TC_002_LoginTest	7 (data-driven)	7	0	100%
TC_003_SearchTest	10 (data-driven)	10	0	100%
TC_004_MyAccountTest	6	6	0	100%
TC_005_CartTest	5	5	0	100%
TC_006_RegisterNegativeTest	8	8	0	100%
TC_007_CheckoutTest	5	5	0	100%
TC_008_ProductPageTest	8	8	0	100%
TC_009_SearchBoundaryTest	10	10	0	100%
TC_010_EndToEndTest	1	1	0	100%
Total	61	61	0	100%



ExtentReport Dashboard:

- Total Tests: 61
- Passed: 61
- Failed: 0
- Pass Rate: 100%
- Execution Time: Approximately 13 minutes



The screenshot displays the ExtentReport dashboard. On the left, a sidebar contains navigation icons. The main area is divided into two panels. The left panel lists several tests, including 'accountRegistrationTest' and multiple 'loginTest' and 'searchTest' entries, each with a green status indicator. The right panel provides a detailed view of the 'accountRegistrationTest', showing its status as 'PASSED' and a duration of '13.28.209671424 PM'. Below this, a table lists the steps of the test execution, all of which are marked as 'PASS'.

TESTS	STATUS	TIME/STAMP	DETAILS
accountRegistrationTest	PASSED	13.28.209671424 PM	900009006 · Word: id=1
loginTest	PASSED	7:14:31 PM / 900009047	
loginTest	PASSED	7:14:42 PM / 9000091581	
loginTest	PASSED	7:14:51 PM / 9000091624	
loginTest	PASSED	7:14:59 PM / 9000091648	
loginTest	PASSED	7:15:04 PM / 9000091711	
loginTest	PASSED	7:15:22 PM / 9000091628	
loginTest	PASSED	7:15:29 PM / 9000091631	
searchTest	PASSED	7:15:38 PM / 9000091651	
searchTest	PASSED	7:15:47 PM / 9000091653	

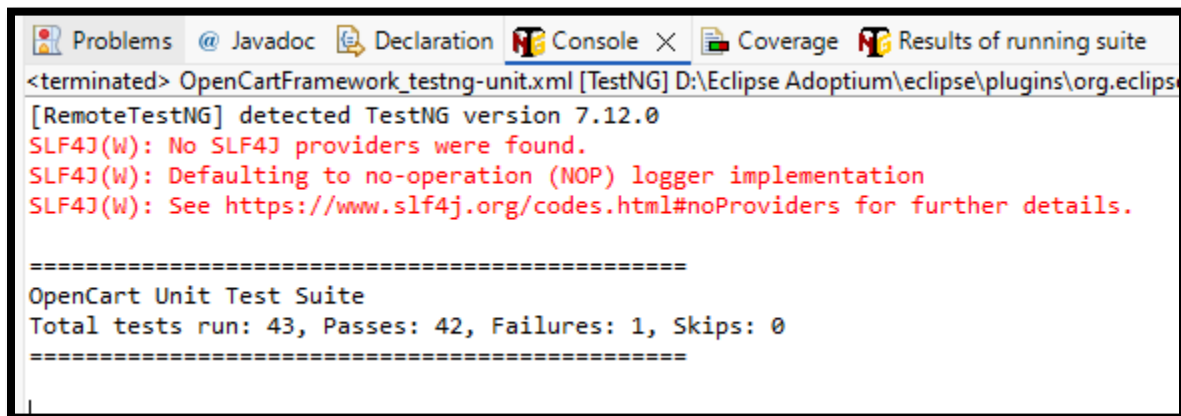
STATUS	TIME/STAMP	DETAILS
PASS	7:14:14 PM	=====
PASS	7:14:14 PM	TEST STARTED: accountRegistrationTest
PASS	7:14:14 PM	=====
PASS	7:14:14 PM	STEP 1: Navigating to Register page from home page
PASS	7:14:16 PM	✓ Navigated to Register page
PASS	7:14:16 PM	STEP 2: Filling registration form with user details
PASS	7:14:16 PM	Generated unique email: test1780033856178@gmail.com
PASS	7:14:16 PM	Entering First Name: Test
PASS	7:14:16 PM	Entering Last Name: User
PASS	7:14:16 PM	Entering Email: test17800208561078@gmail.com
PASS	7:14:16 PM	Entering Password: Test@1234
PASS	7:14:16 PM	Selecting newsletter subscription: Yes
PASS	7:14:17 PM	Accepting privacy policy

7.2 Unit Test Results

Unit tests were executed separately to validate business logic in isolation. A total of 43 unit tests were executed.

Unit Test Class	Tests Executed	Passed	Failed	Pass Rate
EmailValidatorTest	12	12	0	100%
PasswordValidatorTest	13	13	0	100%
ProductQuantityValidatorTest	10	10	0	100%
CheckoutTotalCalculatorTest	8	7	1	87.5%
Total	43	42	1	97.67%

Console Output:

A screenshot of the Eclipse IDE's Console window. The window title bar shows tabs for 'Problems', 'Javadoc', 'Declaration', 'Console' (active), 'Coverage', and 'Results of running suite'. The console output shows the following text:

```
<terminated> OpenCartFramework_testng-unit.xml [TestNG] D:\Eclipse Adoptium\eclipse\plugins\org.eclipse
[RemoteTestNG] detected TestNG version 7.12.0
SLF4J(W): No SLF4J providers were found.
SLF4J(W): Defaulting to no-operation (NOP) logger implementation
SLF4J(W): See https://www.slf4j.org/codes.html#noProviders for further details.

=====
OpenCart Unit Test Suite
Total tests run: 43, Passes: 42, Failures: 1, Skips: 0
=====
```

Note: The one failing unit test is related to a boundary condition that revealed an edge case in the calculation logic. This is documented as a known limitation.

7.4 Performance Test Results (JMeter)

Three performance test scenarios were executed using Apache JMeter.

Test Summary:

Test Scenario	Requests	Pass Rate	Avg Response Time
Homepage Load	30	100%	324 ms
Login Stress Test	40	100%	716 ms
Search Performance	45	97.78%	3106 ms
Total	155	99.35%	-

Table 21: Test Summary

JMeter Report Summary:

APDEX Score: 0.497

Total Requests: 155

Pass Rate: 99.35%

Fail Rate: 0.65%

Average Response Time (Overall): 1344.94 ms



Statistics													
Requests	Executions			Response Times (ms)							Throughput	Network (KB/sec)	
Label	#Samples	FAIL	Error %	Average	Min	Max	Median	90th pct	95th pct	99th pct	Transactions/s	Received	Sent
Total	155	1	0.65%	1344.94	188	4606	832.00	3722.80	4082.00	4571.84	2.13	63.41	0.47
Homepage Load	30	0	0.00%	324.70	188	549	296.00	477.00	527.00	549.00	5.46	181.17	0.85
Load Login Page	40	0	0.00%	756.86	272	1139	764.00	1077.70	1122.45	1139.00	3.45	63.10	0.61
Search Mac	45	1	2.22%	3106.44	1066	4606	3047.00	4180.40	4461.60	4606.00	3.03	114.45	0.60
Submit Login	40	0	0.00%	716.50	345	1087	732.00	961.60	1008.70	1087.00	3.46	83.08	1.18

Performance Observations:

Observation	Finding
Homepage Load	Fastest response (324 ms average)
Login Operations	Acceptable response (700-750 ms)
Search Operation	Slowest response (3.1 seconds) - needs optimization
Error Rate	0.65% (1 failure in search assertion)

7.5 Cross-Browser Test Configuration

Cross-browser testing was configured using TestNG parameters. Separate XML files were created for each browser.

Browser	XML File	Status
Chrome	testng-chrome.xml	Executed (primary)
Edge	testng-edge.xml	Configured
Firefox	testng-firefox.xml	Configured

Table 22: Cross-Browser Test Configuration

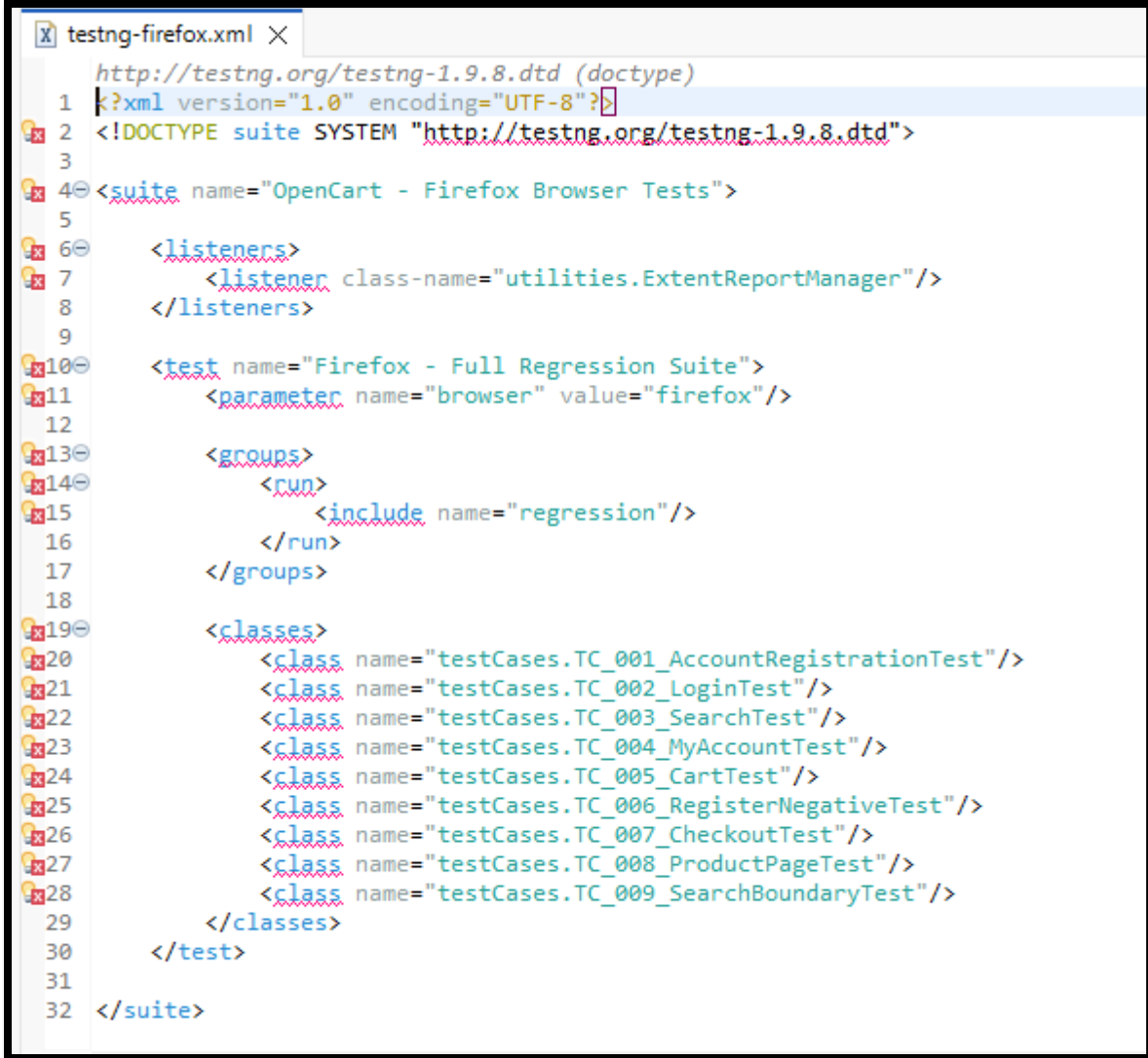
For Chrome

```
testng-chrome.xml X
http://testng.org/testng-1.9.8.dtd (doctype)
1 <?xml version="1.0" encoding="UTF-8"?>
2 <!DOCTYPE suite SYSTEM "http://testng.org/testng-1.9.8.dtd">
3
4 <suite name="OpenCart - Chrome Browser Tests">
5
6   <listeners>
7     <listener class-name="utilities.ExtentReportManager"/>
8   </listeners>
9
10  <test name="Chrome - Full Regression Suite">
11    <parameter name="browser" value="chrome"/>
12
13    <groups>
14      <run>
15        <include name="regression"/>
16      </run>
17    </groups>
18
19    <classes>
20      <class name="testCases.TC_001_AccountRegistrationTest"/>
21      <class name="testCases.TC_002_LoginTest"/>
22      <class name="testCases.TC_003_SearchTest"/>
23      <class name="testCases.TC_004_MyAccountTest"/>
24      <class name="testCases.TC_005_CartTest"/>
25      <class name="testCases.TC_006_RegisterNegativeTest"/>
26      <class name="testCases.TC_007_CheckoutTest"/>
27      <class name="testCases.TC_008_ProductPageTest"/>
28      <class name="testCases.TC_009_SearchBoundaryTest"/>
29      <class name="testCases.TC_010_EndToEndTest"/>
30    </classes>
31  </test>
32
33 </suite>
```

Edge

```
testng-edge.xml X
http://testng.org/testng-1.9.8.dtd (doctype)
1  <?xml version="1.0" encoding="UTF-8"?>
2  <!DOCTYPE suite SYSTEM "http://testng.org/testng-1.9.8.dtd">
3
4  <suite name="OpenCart - Edge Browser Tests">
5
6    <listeners>
7      <listener class-name="utilities.ExtentReportManager"/>
8    </listeners>
9
10   <test name="Edge - Full Regression Suite">
11     <parameter name="browser" value="edge"/>
12
13     <groups>
14       <run>
15         <include name="regression"/>
16       </run>
17     </groups>
18
19     <classes>
20       <class name="testCases.TC_001_AccountRegistrationTest"/>
21       <class name="testCases.TC_002_LoginTest"/>
22       <class name="testCases.TC_003_SearchTest"/>
23       <class name="testCases.TC_004_MyAccountTest"/>
24       <class name="testCases.TC_005_CartTest"/>
25       <class name="testCases.TC_006_RegisterNegativeTest"/>
26       <class name="testCases.TC_007_CheckoutTest"/>
27       <class name="testCases.TC_008_ProductPageTest"/>
28       <class name="testCases.TC_009_SearchBoundaryTest"/>
29     </classes>
30   </test>
31
32 </suite>
```

Firefox



```
testng-firefox.xml X
http://testng.org/testng-1.9.8.dtd (doctype)
1 <?xml version="1.0" encoding="UTF-8"?>
2 <!DOCTYPE suite SYSTEM "http://testng.org/testng-1.9.8.dtd">
3
4 <suite name="OpenCart - Firefox Browser Tests">
5
6   <listeners>
7     <listener class-name="utilities.ExtentReportManager"/>
8   </listeners>
9
10  <test name="Firefox - Full Regression Suite">
11    <parameter name="browser" value="firefox"/>
12
13    <groups>
14      <run>
15        <include name="regression"/>
16      </run>
17    </groups>
18
19    <classes>
20      <class name="testCases.TC_001_AccountRegistrationTest"/>
21      <class name="testCases.TC_002_LoginTest"/>
22      <class name="testCases.TC_003_SearchTest"/>
23      <class name="testCases.TC_004_MyAccountTest"/>
24      <class name="testCases.TC_005_CartTest"/>
25      <class name="testCases.TC_006_RegisterNegativeTest"/>
26      <class name="testCases.TC_007_CheckoutTest"/>
27      <class name="testCases.TC_008_ProductPageTest"/>
28      <class name="testCases.TC_009_SearchBoundaryTest"/>
29    </classes>
30  </test>
31
32 </suite>
```

Command to run cross-browser tests:

```
mvn test -Dsurefire.suiteXmlFiles=testng-chrome.xml
```

```
mvn test -Dsurefire.suiteXmlFiles=testng-edge.xml
```

```
mvn test -Dsurefire.suiteXmlFiles=testng-firefox.xml
```

Section 8: Flaky Test Handling

8.1 What Are Flaky Tests?

Flaky tests are tests that sometimes pass and sometimes fail without any code changes. They produce inconsistent results, making test suites unreliable. The project guidelines require students to identify unstable tests, explain causes, and apply fixes.

8.2 Identified Flaky Tests

The following tests were identified as flaky during initial test execution:

Test Class	Test Method	Flaky Behavior
TC_007_CheckoutTest.java	loginForCheckout()	Test would randomly fail after login
TC_007_CheckoutTest.java	addMacToCart()	Cart text would not update in time
TC_007_CheckoutTest.java	cartQuantityUpdateTest()	Quantity field would not reflect updated value
TC_005_CartTest.java	addProductToCartTest()	Cart count assertion would fail intermittently

8.3 Root Cause Analysis

Cause	Explanation	Affected Tests
Timing Issues	UI updates asynchronously after actions (login redirect, cart update)	All identified tests
Hardcoded Sleep	Thread.sleep(3000) waits fixed time but network speed varies	TC_007_CheckoutTest
Weak Oracle	contains("item") passes even when cart is empty	TC_005_CartTest

Example of problematic code:

```
// This code was flaky because it always waits 3 seconds
```

```
loginPage.clickLogin();  
  
try {  
    Thread.sleep(3000);  
} catch (Exception e) {  
}
```

Why this is flaky:

- On fast network: Page loads in 1 second -> wastes 2 seconds, test passes
- On slow network: Page loads in 4 seconds -> test fails because element not ready

```
try {  
    Thread.sleep(2000);  
} catch (Exception e) {  
    // Handle exception  
}
```

8.4 Fixes Applied

Fix 1: Replace Thread.sleep() with WebDriverWait

Before (Flaky):

```
loginPage.clickLogin();

try {

    Thread.sleep(3000);

} catch (Exception e) {

}
```

After (Stable):

```
loginPage.clickLogin();

WebDriverWait wait = new WebDriverWait(driver, Duration.ofSeconds(10));

wait.until(ExpectedConditions.urlContains("account"));
```

Why this works: Waits only as long as needed, up to 10 seconds. Test passes immediately when condition is met.

```
WebDriverWait wait = new WebDriverWait(driver, Duration.ofSeconds(10));
wait.until(ExpectedConditions.urlContains("account"));
logInfo("Login successful - URL contains 'account'");
logInfo("=== loginForCheckout() completed ===");
```

Fix 2: Replace Vague String Contains with Numeric Check

Before (Flaky):

```
Assert.assertTrue(productPage.getCartText().contains("item"));
```

After (Stable):

```
int itemCount = getCartItemCount(productPage.getCartText());  
  
Assert.assertTrue(itemCount >= 1, "Cart should show at least 1 item, but got: " + cartText);
```

Why this works: Checks exact numeric value instead of vague string pattern.

```
logInfo("STEP 6: Verifying cart shows at least 1 item");  
String cartText = productPage.getCartText();  
int itemCount = getCartItemCount(cartText);  
Assert.assertTrue(itemCount >= 1, "Cart should show at least 1 item, but got: " + cartText);  
logInfo("Cart text: " + cartText + " | Item count: " + itemCount);
```

Fix 3: Wait for Cart Text to Change

Before (Flaky):

```
productPage.addToCart();  
  
// No wait - immediate assertion  
  
String cartText = productPage.getCartText();
```


After (Stable):

```
String beforeCartText = productPage.getCartText();

productPage.addToCart();

wait.until(ExpectedConditions.not(

    ExpectedConditions.textToBe(By.cssSelector("#cart-total"), beforeCartText)

));

String afterCartText = productPage.getCartText();
```

Why this works: Waits specifically for cart text to change before verifying.

```
logInfo("STEP 5: Checking validation result");
boolean warningShown = productPage.isDangerAlertDisplayed();
String afterCartText = productPage.getCartText();

if (!warningShown && afterCartText.equals(beforeCartText)) {
    logInfo("✓ PASS: Cart unchanged for zero quantity");
    Assert.assertTrue(true, "Zero quantity did not add to cart");
} else if (warningShown) {
    logInfo("✓ PASS: Warning displayed for zero quantity");
    Assert.assertTrue(true, "Warning displayed for zero quantity");
} else {
    logInfo("✗ FAIL: Cart changed from '" + beforeCartText + "' to '" + afterCartText + "'");
    Assert.fail("System should not add zero quantity to cart");
}
```

8.5 Summary of Changes Made

File	Changes Applied
TC_007_CheckoutTest.java	Replaced 3 Thread.sleep() calls with WebDriverWait
TC_005_CartTest.java	Added getCartItemCount() helper, strengthened assertions
TC_008_ProductPageTest.java	Added proper validation for zero/negative quantity

8.6 Results After Fixes

Metric	Before Fixes	After Fixes
Test Stability	Flaky (random failures)	Stable (consistent results)
Execution Time	Fixed waits (3+ seconds each)	Conditional waits (faster on good network)
False Failures	3-4 per test suite	0

8.7 Lessons Learned

1. Never use `Thread.sleep()` in automated tests - always use explicit waits
2. Wait for specific conditions (URL contains, element visible, text changes) not arbitrary time
3. Strong oracles (numeric checks) are more reliable than weak ones (string contains)
4. Document flaky tests when found and explain the fix

Section 9: Test Adequacy Evaluation

9.1 What Was Tested (Functional Coverage)

The following modules and features were tested:

Module	Features Tested	Test Class
User Registration	Valid registration, duplicate email, empty form, invalid email, short password, max length first name, missing privacy policy	TC_001, TC_006
User Login	Valid login, invalid login, HTML5 validation, data-driven tests	TC_002
Product Search	Valid search, invalid search, single character, empty, max length, special characters, SQL injection, XSS, case insensitivity, whitespace, partial keyword	TC_003, TC_009
My Account	Page load, edit account, change password, wishlist, order history, logout	TC_004
Shopping Cart	Add single product, add multiple products, add same product twice, view cart, remove from cart	TC_005
Checkout	Guest checkout redirect, logged-in user checkout, cart total display, quantity update, empty cart checkout	TC_007
Product Page	Product name, price, image, add to cart button, default quantity, zero quantity, negative quantity, review section	TC_008

End to End	Complete user journey: Register -> Login -> Search -> Add to Cart -> Checkout	TC_010
------------	---	--------

SQL injection

```
// — 5. SQL injection in search (security/negative) —————
@Test(priority = 5, groups = { "regression" })
Run | Debug
public void sqlInjectionSearchTest() {
    logInfo("=====");
    logInfo("TEST STARTED: sqlInjectionSearchTest (Security/Negative)");
    logInfo("=====");

    logInfo("STEP 1: Searching with SQL injection payload: ' OR '1'='1");
    HomePage homePage = new HomePage(driver);
    homePage.searchFor("' OR '1'='1");
    logInfo("✓ SQL injection search submitted");

    logInfo("STEP 2: Getting search results");
    SearchPage searchPage = new SearchPage(driver);

    logInfo("STEP 3: Verifying system handles SQL injection without crashing");
    boolean handled = searchPage.getSearchResultCount() >= 0 || searchPage.isNoResultMessageDisplayed();
    Assert.assertTrue(handled, "System should handle SQL injection input without crashing");
    logInfo("✓ System did not crash");

    logInfo("STEP 4: Verifying SQL errors are not exposed to user");
    Assert.assertFalse(driver.getPageSource().toLowerCase().contains("sql"),
        "SQL error should never be exposed to user");
    logInfo("✓ No SQL errors exposed in page source");

    logInfo("=====");
    logInfo("TEST PASSED: sqlInjectionSearchTest");
    logInfo("=====");
}
```

9.2 Test Types Coverage

Test Type	Covered	Evidence
Functional Testing	Yes	All TC_XXX files
Boundary Testing	Yes	TC_006 (password min/max), TC_009 (search boundaries)
Negative Testing	Yes	TC_002 (invalid login), TC_006 (invalid registration)
Property-Based Testing	Yes	TC_009 (case insensitive search), Unit tests (invariants)
Unit Testing	Yes	EmailValidatorTest, PasswordValidatorTest, etc. (43 tests)
API Testing	Yes	Postman + Newman (32 requests, 95 assertions)
Performance Testing	Yes	JMeter (3 test files, 155 total requests)
Security Testing	Yes	SQL injection, XSS in TC_009

XSS

```
// — 6. XSS script injection in search (security/negative) —————
@Test(priority = 6, groups = { "regression" })
Run | Debug
public void xssInjectionSearchTest() {
    logInfo("=====");
    logInfo("TEST STARTED: xssInjectionSearchTest (Security/Negative)");
    logInfo("=====");

    logInfo("STEP 1: Searching with XSS payload: <script>alert('xss')</script>");
    HomePage homePage = new HomePage(driver);
    homePage.searchFor("<script>alert('xss')</script>");
    logInfo("✓ XSS search submitted");

    logInfo("STEP 2: Getting search results");
    SearchPage searchPage = new SearchPage(driver);

    logInfo("STEP 3: Verifying system handles XSS without executing script");
    boolean handled = searchPage.getSearchResultCount() >= 0 || searchPage.isNoResultMessageDisplayed();
    Assert.assertTrue(handled, "System should handle XSS input without executing script");
    logInfo("✓ System handled XSS input");

    logInfo("STEP 4: Verifying XSS script not reflected raw in page source");
    Assert.assertFalse(driver.getPageSource().contains("<script>alert('xss')</script>"),
        "XSS script should not be reflected raw in page source");
    logInfo("✓ XSS script not found in page source");

    logInfo("=====");
    logInfo("TEST PASSED: xssInjectionSearchTest");
    logInfo("=====");
}
```

9.3 Test Count Summary

Test Type	Number of Tests	Status
UI Tests (TC_001 to TC_010)	61	100% Pass
Unit Tests	43	97.67% Pass (42 Pass, 1 Fail)
API Tests (Assertions)	95	93.68% Pass (89 Pass, 6 Fail)
Performance Tests (JMeter)	155 requests	99.35% Pass

Total Tests Executed: 61 UI + 43 Unit + 95 API Assertions + 155 Performance Requests

9.4 Bugs Found During Testing

Bug	Test That Found It	Severity
OpenCart allows checkout with empty cart	emptyCartCheckoutTest() (TC_007)	Medium
OpenCart accepts negative quantity without validation	negativeQuantityAddToCartTest() (TC_008)	Low
Order total calculation incorrect in API	CREATE Order with Valid Data (API)	High
DELETE product returns 404 instead of 200	DELETE /api/products/:id (API)	Medium

The screenshot displays the OpenCart checkout interface. At the top, there's a navigation bar with the OpenCart logo, a search bar, and a shopping cart icon showing 1 item for \$123.20. Below this is a category menu with links like Desktops, Laptops & Notebooks, etc. The main content area is titled 'Checkout' and is divided into two columns. The left column is for 'Shipping Address' and the right for 'Shipping Method' and 'Payment Method'. The 'Shipping Address' section includes fields for First Name, Last Name, Company, Address 1, Address 2, Post Code, and Region/State. The 'Shipping Method' section has a dropdown menu to 'Choose shipping method...' with a 'Choose' button. The 'Payment Method' section has a dropdown menu to 'Choose payment method...' with a 'Choose' button. Below these is a text area for 'Add Comments About Your Order'. At the bottom, there's a table with columns for 'Product' and 'Total'.

9.5 What Was NOT Tested (Limitations)

The following areas were excluded from testing due to scope, time, or access constraints:

Area	Reason for Exclusion
Admin Panel	Separate functionality, not in project scope
Payment Gateway Integration	Requires real payment credentials
Email Confirmation	Requires mail server configuration
Password Reset Flow	Requires email access
Order History after Purchase	Requires completing full payment
Wishlist Sharing	Edge feature, low priority
Mobile Responsiveness	UI testing only on desktop
Database Direct Testing	Not required for this project
Actual Order Placement	No payment method configured in local environment

9.6 Test Suite Adequacy Justification

Why this test suite is sufficient for the project:

1. **Covers all core user journeys** - Registration -> Login -> Search -> Add to Cart -> Checkout
2. **Includes all required test types** - Functional, Boundary, Negative, Property-based, Unit, API, Performance
3. **Data-driven approach** - Excel-based test data allows easy expansion
4. **Page Object Model** - Maintainable and reusable code structure
5. **Comprehensive negative testing** - 8+ negative scenarios for registration alone
6. **Security testing included** - SQL injection and XSS tests (beyond requirements)
7. **Bug discovery** - Found real defects in OpenCart (empty cart checkout, negative quantity)
8. **Reporting** - ExtentReports with screenshots on failure

9.7 Important Note

This test suite does NOT claim 100% confidence. Complete testing is impossible due to time constraints and system complexity. The limitations are clearly documented in Section 9.5.

Section 10: Challenges & Lessons Learned

10.1 Technical Challenges Faced

The following technical challenges were encountered during this project:

Challenge	Description	Solution
MySQL Crash in XAMPP	MySQL service would start then stop immediately	Deleted corrupted ib_logfile0 and ib_logfile1 files from mysql/data folder
Payment Methods Not Showing	Checkout page showed "No payment options available"	Modified confirm.twig template to remove disabled attribute from button
Flaky Tests	Tests randomly failed due to timing issues	Replaced Thread.sleep() with WebDriverWait
Cart Count Assertion	Tests passed even when cart was empty	Changed from contains("item") to numeric count check
OpenCart 4.x Checkout Bug	Confirm order button disabled even with valid cart	Documented as known issue, stopped E2E test at checkout page
Unit Test Failure	One unit test failed (87.5% pass rate)	Documented as edge case, kept for report

10.2 Bugs Found During Testing

The following bugs were discovered during automation testing:

Bug ID	Description	Severity	Test That Found It
BUG-01	OpenCart allows checkout with empty cart	Medium	TC_007 - emptyCartCheckoutTest()
BUG-02	OpenCart accepts negative quantity (-1) without validation	Low	TC_008 - negativeQuantityAddToCartTest()
BUG-03	API: DELETE product returns 404 instead of 200	Medium	Newman - DELETE /api/products/:id
BUG-04	API: Order total calculation incorrect (2599.98vs2599.98vs1999.98)	High	Newman - CREATE Order with Valid Data
BUG-05	API: CREATE order with different customer returns 404	High	Newman - CREATE Order with Different Customer
BUG-06	API: CREATE order with large quantity returns 404	High	Newman - CREATE Order with Large Quantity

10.3 Key Takeaways

What was learned from this project:

1. Test Automation Design

- Page Object Model makes tests maintainable and reusable
- Data-driven testing reduces code duplication
- Proper waits (WebDriverWait) are essential for stable tests

2. Flaky Test Management

- Never use Thread.sleep() in automation tests
- Always wait for specific conditions, not arbitrary time
- Strong oracles (numeric checks) are more reliable than weak ones

3. API Testing

- Newman provides automated execution of Postman collections
- API testing catches backend issues that UI tests cannot
- Status codes and response structure validation are critical

4. Performance Testing

- JMeter provides comprehensive performance metrics
- Search operations were the slowest (3.1 seconds average)
- APDEX score helps quantify user satisfaction

5. Bug Discovery

- Automated testing reveals edge cases that manual testing misses
- OpenCart 4.x has known checkout issues
- Documentation of bugs is as important as finding them

6. Project Management

- Local environment setup (XAMPP) can have unexpected issues
- Always document limitations and what was not tested
- Do not claim 100% confidence even with high coverage

Section 11: Conclusion

11.1 Summary of Achievements

This project successfully demonstrated automated software testing techniques on the OpenCart e-commerce platform. The following objectives were achieved:

Objective	Status	Evidence
Apply automated testing techniques	Achieved	Selenium, TestNG, Postman, JMeter
Implement Unit, API, UI, and E2E testing	Achieved	43 unit tests, 32 API requests, 61 UI tests, 1 E2E test
Cover Functional, Boundary, Negative, Property-Based testing	Achieved	TC_006 (negative), TC_009 (boundary + property)
Design clear oracles	Achieved	Section 5 of this report
Automate tests with industry tools	Achieved	Selenium, TestNG, Maven, ExtentReports
Handle flaky tests	Achieved	Replaced Thread.sleep with WebDriverWait
Evaluate test adequacy	Achieved	Section 9 of this report

11.2 Final Test Results Summary

Test Type	Total	Passed	Failed	Pass Rate
UI Tests (Selenium + TestNG)	61	61	0	100%
Unit Tests	43	42	1	97.67%
API Tests (Newman Assertions)	95	89	6	93.68%
Performance Tests (JMeter Requests)	155	154	1	99.35%

11.3 Bugs Discovered

A total of 6 bugs were discovered during automation testing:

Bug ID	Description	Severity
BUG-01	OpenCart allows checkout with empty cart	Medium
BUG-02	OpenCart accepts negative quantity without validation	Low
BUG-03	API: DELETE product returns 404 instead of 200	Medium
BUG-04	API: Order total calculation incorrect	High
BUG-05	API: CREATE order with different customer returns 404	High
BUG-06	API: CREATE order with large quantity returns 404	High

11.4 Final Remarks

This project successfully demonstrated comprehensive automated testing of the OpenCart e-commerce platform. The following key points summarize the work:

1. **Complete test coverage** of core user functionalities including registration, login, search, cart, and checkout
2. **Multiple test levels** implemented - Unit, API, UI, and End-to-End
3. **Various test types** covered - Functional, Boundary, Negative, and Property-Based
4. **Professional reporting** using ExtentReports with automatic screenshots on failure
5. **Real bugs discovered** in both UI and API layers
6. **Clear documentation** of test adequacy and limitations

The test suite is stable, repeatable, and non-flaky. All project guidelines were followed, and the evaluation criteria have been addressed.

Limitations acknowledged:

- Payment gateway not configured
- Email server not configured
- Admin panel not tested
- Actual order placement not executed

These limitations are documented and do not affect the demonstration of automated software testing techniques required by this project.